

3. 记忆处理流程

记忆处理将一次次 AI 交互转化为可长期复用的用户画像、偏好、业务实体、事件、Agent 案例、工具模式和技能经验。它解决的问题是：AI 应用不能只依赖静态资料，还需要从真实服务和业务执行中持续学习。

Session 创建和交互

- > add_message 写入用户/助手/工具消息
- > used 记录本轮引用过的上下文或技能
- > commit_async 立即归档当前消息
- > 后台任务生成 archive L0/L1 摘要
- > MemoryExtractor 抽取长期记忆候选
- > 向量预筛 + MemoryDeduplicator 去重/合并
- > 写入 viking://user/memories 或 viking://agent/memories
- > 更新 active_count 与 .done/.failed 状态

3.1 Session 生命周期

阶段	关键动作	技术要点
创建	SessionService 创建 session 目录和 .meta.json	URI 形如 viking://session/{user_space}/{session_id}
交互	add_message 追加 TextPart、ContextPart、ToolPart 等消息片段	消息保存为 JSONL，便于增量读取与审计
引用记录	used 记录本轮使用的资源、记忆或技能 URI	commit 后可更新 active_count，支持热度治理
提交	commit_async 将当前消息移入 history/archive_xxx	同步阶段快速返回 task_id，异步阶段处理摘要和记忆
归档完成	写入 .abstract.md、.overview.md、.done	失败时写 .failed.json，并阻断后续依赖归档以避免链路错乱

3.2 两阶段提交机制

Session commit 分为同步和异步两阶段。同步阶段不调用 LLM：它只递增 compression_index，将消息写入 archive_xxx/messages.jsonl，清空当前消息，并立即返回 task_id。异步阶段再生成 archive 摘要、抽取长期记忆、更新活跃度和写完成标记。这种设计让对话主链路不被模型摘要耗时拖慢。

提交阶段	输入	输出	失败处理
Phase 1 同步归档	当前未提交消息、使用记录快照	archive URI、task_id、当前消息清空	若无消息则返回 archived=false
Phase 2 异步压缩	archive messages、上一归档 overview	.abstract.md、.overview.md、.meta.json	写 .failed.json，后续归档可检测阻断
长期记忆抽取	归档消息、工具调用、上下文引用	CandidateMemory 列表	抽取失败不影响已归档原始记录
去重与合并	候选记忆 + 相似历史记录	create / merge / skip / delete 决策	保留可审计的候选与处理日志